

Answers for Week 6

1. Mean Faces



In this exercise we will use the [CelebA](#) dataset of celebrity images as our data source. Examples are shown above.

You can find it at: `/dtu/projects/02613_2025/data/celeba/`. We have also created smaller subsets for fast testing. To begin with use the 'celeba_200.npy' file for testing and then work your way up to at least the 'celeba_100K.npy' version.

Each image is 128×128 RGB pixels. They are stored in an $N \times 128 \times 128 \times 3$ array, such that `array[i]` is a $128 \times 128 \times 3$ image. The goal is to compute the mean (i.e., average) face by summing all the images and then dividing by the number of images. Since CelebA contains over 200,000 images, we want the sum to be parallel, i.e., we want a parallel reduction.

In order to get good performance, we need to minimize data transferred between processes. We can do this by creating an array of shared memory and then performing the reduction in place. The template below sets up a shared array and performs a 'dummy' operation. Adapt it for this exercise.

```
1 import ctypes
2 import multiprocessing as mp
3 import sys
4 from time import perf_counter as time
5 import numpy as np
6 from PIL import Image
7
8
9 def init(shared_arr_):
10     global shared_arr
11     shared_arr = shared_arr_
```

```

12
13
14 def tonumpyarray(mp_arr):
15     return np.frombuffer(mp_arr, dtype='float32')
16
17
18 def reduce_step(args):
19     b, e, s, elemshape = args
20     arr = tonumpyarray(shared_arr).reshape((-1,) + elemshape)
21     # Change the code below to compute a step of the reduction
22     # -----8<-----
23     arr[b:e:s] = 1.0 - arr[b:e:s] # <-- Dummy op. Replace with correct
24
25
26 if __name__ == '__main__':
27     n_processes = 1
28     chunk = 2
29
30     # Create shared array
31     data = np.load(sys.argv[1])
32     elemshape = data.shape[1:]
33     shared_arr = mp.RawArray(ctypes.c_float, data.size)
34     arr = tonumpyarray(shared_arr).reshape(data.shape)
35     np.copyto(arr, data)
36     del data
37
38     # Run parallel sum
39     t = time()
40     pool = mp.Pool(n_processes, initializer=init, initargs=(shared_arr
41         ,))
42
43     # Change the code below to compute a step of the reduction
44     # -----8<-----
45     pool.map(reduce_step,
46         [(i, i + chunk, 1, elemshape) for i in range(0, len(arr),
47             chunk)],
48         chunksize=1)
49
50     # Write output
51     print(time() - t)
52     final_image = arr[0]
53     # final_image /= len(arr) # For mean
54     Image.fromarray(
55         (255 * final_image.astype(float)).astype('uint8')
56     ).save('result.png')

```

1. **Autolab** Modify the provided example to compute the first step of a parallel reduction, i.e., every second element is summed with its neighbor. For example, after this step, an input of $[1, 2, 3, 4]$ should be $[1 + 2, 2, 3 + 4, 4] = [3, 2, 7, 4]$.
Hint: for testing, make some dummy data like:

```
1 import numpy as np
2 arr = np.arange(10) # 0 to 9
3 arr = arr.astype('float32')
4 arr = arr[:, None, None, None] # (10, 1, 1, 1)
5 np.save('dummydata.npy', arr)
```

Then, comment out the lines saving the image and instead just print the array to manually inspect the results.

Answer: Autolab exercise - answer omitted

2. **Autolab** Further modify the example to compute the full reduction. Remember to divide by the number images at the end to compute the mean. Hint: each step of the reduction should be a separate call to `pool.map`.

Answer: The final face should look something like this:



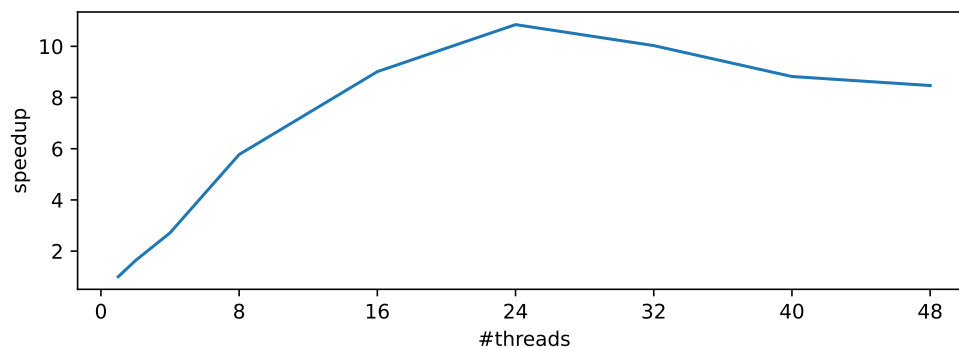
Autolab exercise - Code omitted.

3. Play with the chunk size (the chunk variable in the code). What value gives the best performance for you? Note, this will depend on the size of the subset you use, so make sure to try the program for at least the 100,000 subset.

Answer: Just experimenting with some different values, I found that a chunk size of 64 gave me good performance. The remaining experiments have therefore been done with this chunk size.

4. Run your program as a batch job for varying number of processes and create a speedup plot. Run the program for at least the 100,000 subset. For consistent results, you should run the program 3 times and use the average time. What do you see? Is anything surprising? Hint: **this will take time** for the 100K subset or above. While you wait, start the next exercise.

Answer: Your exact plot will depend on the hardware. For a Xeon Gold 6342 with 48 available threads, I get:

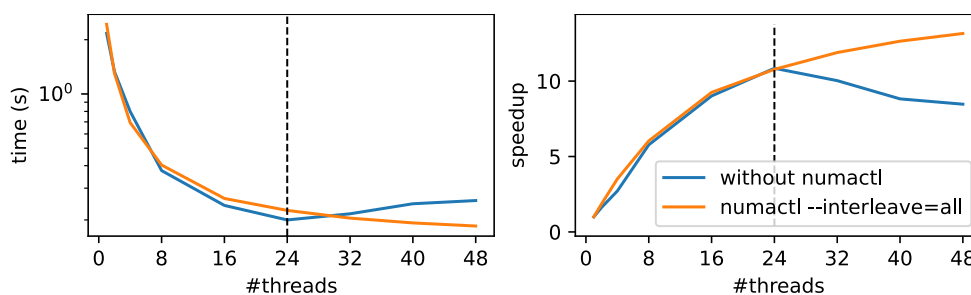


While it looks promising for the first 24 cores, the speed then *decreases* from there! That is surprising - we would expect it to keep improving. Let's try to remedy that.

5. **Autolab** Repeat the previous exercise, but run Python under `numactl --interleave=all`. What changed? Hint: see lecture slides for examples with `numactl`.

Answer: We run the program as: `numactl --interleave=all python reduction.py <...>`.

Comparing the speedup plots, we get:



Autolab exercise - Comments omitted.

6. Compare your fastest runtime with `np . sum`. Do you manage to compute the sum faster? If so, by how much?

Answer: The fastest runtime should be faster than just using `np . sum`. However, not by too much: 2-4x is fine. Python has a lot of overhead, so it is very difficult to beat the optimized NumPy code.
